

Uniform Ordering Optimization

Prediction to Prescription:

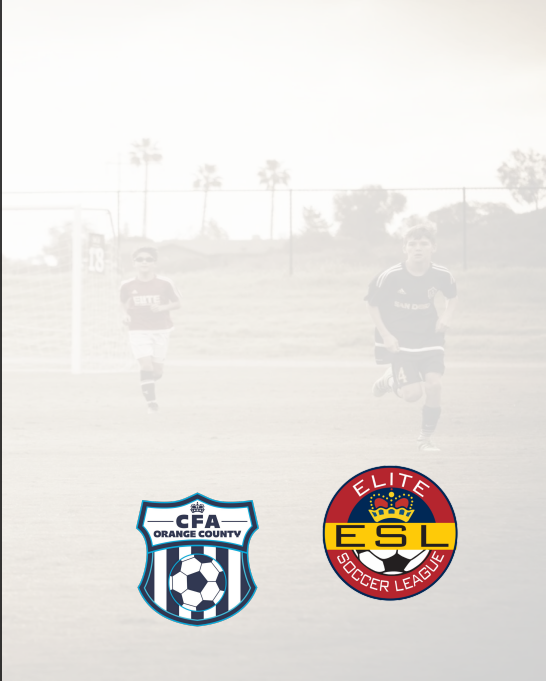
Building an AI Agent for Business Decisions

S. Ganteda · D. Affleck · S. Shenoy

F. Macias Peñalosa · A. Gupta

Team 7 · MGMT 590-037 · Summer 2026

Daniels School of Business — Purdue University



1. The Business Problem
2. Decision-First Formulation
3. Data: Source & Preparation
4. Agent Architecture
5. What Each Agent Does
6. Prediction Methodology
7. Optimization Engine
8. Two-Period Joint Model
9. Adaptability & Benchmarking
- 10. Live Agent Demo**
11. Results & Managerial Takeaways
12. Limitations & Future Work

15 minutes + 5 min Q&A

The Business Problem

Client: Elite Soccer League / CFA — Orange County, CA

The Decision: How many uniforms to order per size and product type—*before* demand is known.

Why It Is Hard:

2,000–3,000 orders/season across **14 sizes** × **3 categories**

Newsvendor problem—commit before demand realized

Asymmetric costs—stockouts up to 5× costlier

Jersey lifecycle—Adidas models discontinued every 2 yrs; Year 2 stock has *no reorder path*

Budget coupling—all sizes share fixed seasonal budget



Cost Asymmetry (Year 2)

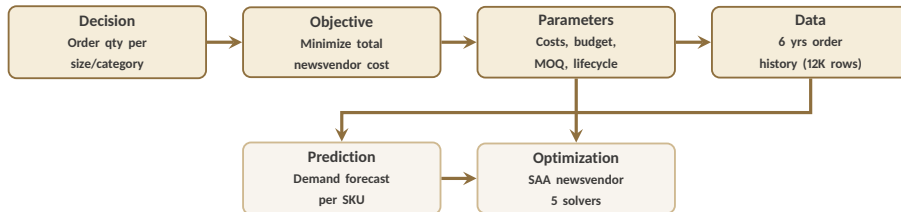
Category	Over.	Under.	CR
Tops (jerseys)	\$15	\$60	0.80
Bottoms (shorts)	\$8	\$25	0.76
Socks	\$1	\$5	0.83

$CR = c_U / (c_U + c_O) = \text{optimal service level}$ Higher CR $\Rightarrow$ order *above* median

Current Process

25 years of “gut feel”—systematic over- and under-ordering

Decision-First Formulation



Objective Function:

$$\min_{\mathbf{q}} \sum_i \left[c_i q_i + \frac{1}{S} \sum_s (o_i [q_i - d_{is}]^+ + u_i [d_{is} - q_i]^+) \right]$$

Constraints:

Budget: $\sum_i c_i q_i \leq \$8,000$ per season

Min. order qty: $q_i \geq 6$ per size (Adidas MOQ)

Non-negativity: $q_i \geq 0$

Source: CFA Squarespace order export

Proprietary data, authorized by league president

Scale:

Raw: 90,010 rows × 52 columns (2017–2026)

Cleaned: 12,209 line items × 10 columns

Key Columns:

order_date, season, year, product_category, gender_age, size, quantity, unit_price

Cleaning Challenges:

- Size embedded in free-text variant strings
- Product names inconsistent across seasons
- Lifecycle year inferred from jersey model
- Interleaved order-level and line-item rows

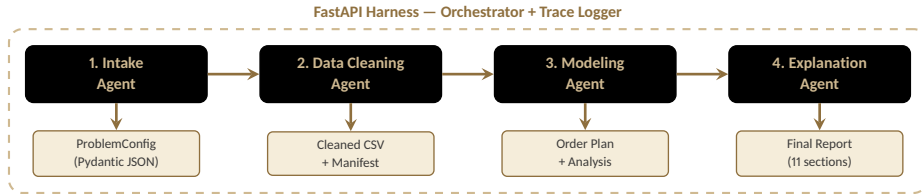
Cleaned Data Profile

Metric	Value
Records	12,209
Date range	2020–2026
Product categories	3 (top, bottom, socks)
Size codes	14 (YXS–AXL, WXS–WXL)
Seasons	4 (fall, winter, spring, CFA)
Null rate	0% after cleaning

Train / Test Split

Chronological (no data leakage)
Training: historical seasons (2020–2023)
Testing: backtested against held-out period
Agent auto-determines optimal split from problem config

Agent Architecture: 4-Stage LLM Pipeline



Design Principles:

- LLMs at *judgment* touchpoints only (3 calls)
- Deterministic Python for all numerics
- Sandboxed code execution
- File-based state passing (auditable, replayable)

Guardrails:

- Pydantic schema validation at every handoff
- Budget constraint enforced structurally
- Structured summaries, not free-form text
- Full JSON trace with timing & token usage

What Each Agent Does

1. Intake Agent (LLM #1)

In: Problem description + CSV **Out:** ProblemConfig JSON

Tools: data inspection (peek, sample, describe)

Asks clarifying Qs grounded in actual data; saves Pydantic-validated config

2. Data Cleaning Agent (LLM #2)

In: Raw CSV + ProblemConfig **Out:** Cleaned CSV + summary

Tools: data inspection + sandboxed code execution

Generates its own Python cleaning code—fully generic, no hardcoded logic

3. Modeling Agent (Tool Orchestration)

In: Cleaned CSV + ProblemConfig **Out:** Order plan

Tools: load_data, run_prediction, run_optimization, validate, sensitivity, baseline

Critically inspects data *before* modeling; tunes hyperparameters

4. Explanation Agent (LLM #3)

In: All run artifacts **Out:** 11-section business report

Tools: read_file, list_files, save_report

Writes for dual audience—instructor and business executive

Models Compared (6 total):

XGBoost—gradient boosting, tunable depth/LR/trees

LightGBM—leaf-wise growth, fast training

Ridge—linear baseline for comparison

Each $\times$ **PTO-adjusted** variant (6 total)

Predict-then-Optimize (PTO):

Shifts predictions by newsvendor critical ratio:

$$q^* = \hat{D} \cdot (1 + (CR - 0.5) \cdot \text{strength})$$

Intentionally biases toward the cost-optimal quantile.

Features:

Auto-engineered: label-encoded categoricals, year trend, season. Agent adds domain features via code.

Output Per SKU:

Point estimate, P10/P50/P90 bounds, feature importance, decision cost

Why Newsvendor Cost, Not RMSE?

Two models with *identical* RMSE can yield very different ordering costs.

The newsvendor penalizes errors **asymmetrically**: under-prediction (stockout) costs up to $5\times$ more than over-prediction.

$\Rightarrow$ We rank models by the **realized newsvendor cost** of the orders they imply.

Following Elmachtoub & Grigas (2022)

Tuning Levers

Agent can adjust: tree depth, learning rate, estimator count, PTO strength, train/test year split

Formulation: Sample Average Approximation (SAA) of the multi-product newsvendor

$$\min_{\mathbf{q}} \sum_{i=1}^N \left[c_i q_i + \frac{1}{S} \sum_{s=1}^S (o_i(q_i - d_{is})^+ + u_i(d_{is} - q_i)^+) \right]$$

Five Solvers Benchmarked:

1. **SLSQP**—sequential quadratic programming
2. **L-BFGS-B**—quasi-Newton, box-constrained
3. **PGD**—projected gradient descent
4. **SGD**—stochastic gradient descent
5. **Adam**—adaptive moment estimation

Multiple random restarts to handle non-convexity.

Constraint Handling:

Budget: hard constraint via SLSQP; projection for first-order

MOQ: floor projection ($q_i \geq 6$)

Non-negativity: box bounds

Agent Verification:

All solvers feasible? Budget binding or slack?

Realized cost vs. expected cost

Agent plan vs. naive baseline

Two-Period Joint Optimization (Model B)

The Jersey Lifecycle Problem:

Adidas models (e.g., Tiro 25) produced for ~ 2 years. Year 2 stockouts mean sourcing discontinued jerseys at $5 \times$ cost or losing sales.

Key Insight:

Year 1 surplus carries to Year 2 at near-zero cost—a **free hedge** against Year 2's extreme underage penalty.

Joint Formulation:

$$\min_{q_1, q_2} \mathbb{E} \left[\sum_s (u_1[D_1 - q_1]^+ + o_2[q_2 + I_s - D_2]^+ + u_2[D_2 - q_2 - I_s]^+) \right]$$

where $I_s = \max(q_1 - D_1, 0)$ = carryover inventory

Model A vs. Model B

	A	B
Scope	Independent	Joint
Periods	1 at a time	Both linked
Carryover	Ignored	Modeled
Y2 hedge	None	Built-in
Solver	Per-period	SAA joint

Why Joint Matters

Independent solving *misses* the carryover hedge. Joint model can deliberately over-order in Year 1 (cheap overage) to reduce Year 2 risk (expensive underage).

$\Rightarrow$ "Cost of myopia" quantified

Re-Run Only (Config Change)

New constraint ("Add a 500 unit sustainability cap")
Changed costs ("Rush shipping now $2 \times$ more")
Different budget ("12K not 8K")
New season of data (drop in updated CSV)
Shifted objective ("Maximize service level")

Benchmark Protocol:

Feed instructor's problem as a description file
Pipeline runs end-to-end automatically
Agent's recommendation compared to known optimal
Discrepancies analyzed in Explanation report

Needs Re-Engineering

Fundamentally different problem class (e.g., scheduling)
New data source not in existing dataset
Structurally new constraint types (tiered discounts)

How Changes Propagate:

User edits ProblemConfig (via chat or JSON)
→ Pipeline re-runs automatically
→ New order plan in ~ 10 minutes



Live Agent Demo

1. Upload CSV and problem description
2. Brainstorm with the Intake Agent
3. Approve the ProblemConfig
4. Watch: Data Cleaning → Modeling → Explanation
5. View the final report with optimized order plan

1. **The agent produces a budget-feasible, cost-minimizing order plan**—something the current “gut feel” process cannot guarantee.
2. **Focus on youth jerseys**—they drive the majority of spend and forecasting risk. Adult/women sizes are often set by MOQ.
3. **The budget is the binding constraint**, not the forecast model. If the budget is too tight, no algorithm can prevent stockouts.
4. **Cost estimates matter more than demand accuracy**—the plan is sensitive to underage cost assumptions and nearly flat to overage.
5. **Re-run when conditions change**: new cost data, updated budget, new season of orders, jersey model transition.
6. **The system is generic**—the same pipeline can optimize any newsvendor-class inventory problem, not just soccer uniforms.

Current Limitations:

Size-level demand inherently noisy—tail spikes not predictable from history alone

Sparse data for rare sizes (AXL, WXL)—MOQ dominates

Pipeline runtime ~ 10 min (optimizer-dominated)

LLM output quality varies between runs

Future Improvements:

Registration/roster data as a leading demand indicator

Hierarchical models to borrow strength across sizes

Carryover-net ordering as a first-class constraint

End-to-end prescriptive model on newsvendor loss

Interactive config editor in web UI

Adaptable

Any change expressible
in ProblemConfig

Transparent

Every agent writes a
structured trace

Testable

Benchmark problems
fed as description files

Questions?

Thank You

S. Ganteda · D. Affleck · S. Shenoy · F. Macias Peñalosa · A. Gupta

MGMT 590-037 — AI-Enhanced Optimization — Summer 2026

Daniels School of Business — Purdue University

